

Inheritance in Java

A Comprehensive Guide

Slide 1: Introduction

What is Inheritance in Java?

- Inheritance allows a class to acquire fields and methods from another class
- Uses the `extends` keyword
- Java supports **single inheritance** only (one direct parent)
- Interfaces provide multiple inheritance of behavior

```
class Animal {                                // Parent class
    void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {                    // Child class
    void bark() {
        System.out.println("Barking...");
    }
}
```

Slide 2: Why Use Inheritance?

Benefits:

- **Code Reusability** — Common logic in the parent
- **Extensibility** — Add features without changing existing code
- **Polymorphism** — Treat derived objects as base type
- **Logical Hierarchy** — Models real-world relationships



Slide 3: The `extends` Keyword

Basic syntax:

```
class Parent {  
    // Fields and methods  
}  
  
class Child extends Parent {  
    // Inherits all accessible members from Parent  
    // Can add new fields and methods  
    // Can override existing methods  
}
```

- Java supports **only single inheritance** — one class can extend only one parent
- All classes implicitly extend `Object`
- `Object` provides `toString()`, `equals()`, `hashCode()`, etc.

Slide 4: The `Object` Class

Every class inherits from `Object`:

```
class MyClass {  
    // Implicitly: class MyClass extends Object  
}  
  
// Inherited methods:  
// toString()      - string representation  
// equals()        - object equality  
// hashCode()      - hash code for hashing  
// getClass()      - runtime class information  
// clone()         - object cloning  
// finalize()      - cleanup before GC (deprecated)
```

```
MyClass obj = new MyClass();  
System.out.println(obj.toString()); // MyClass@1a2b3c  
System.out.println(obj.hashCode()); // 1234567
```

Slide 5: Access Modifiers and Inheritance

Visibility rules:

Modifier	Same Class	Same Package	Subclass (diff pkg)	Anywhere
<code>private</code>	✓	✗	✗	✗
<code>default</code>	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗

Modifier	Same Class	Same Package	Subclass (diff pkg)	Anywhere
<code>public</code>	✓	✓	✓	✓

```
class Parent {
    private int secret;           // Not inherited
    int packagePrivate;          // Inherited in same package
    protected int family;        // Inherited by subclass
    public int open;              // Inherited everywhere
}
```

Slide 6: Inheriting Fields

Fields are inherited but not overridden:

```
class Vehicle {
    protected String brand = "Unknown";
}

class Car extends Vehicle {
    String model = "Camry";

    void show() {
        System.out.println(brand); // Inherited from Vehicle
        System.out.println(model); // Own field
    }
}

Car c = new Car();
c.brand = "Toyota"; // Accessing inherited field
System.out.println(c.brand); // Toyota
```

Fields are **hidden** (not overridden) — if child declares same name, both exist.

Slide 7: The `super` Keyword

Accessing parent members from child:

```
class Parent {
    int value = 10;

    void display() {
        System.out.println("Parent: " + value);
    }
}
```

```
class Child extends Parent {
    int value = 20;    // Hides Parent.value

    void display() {    // Overrides Parent.display()
        super.display();    // Call parent method
        System.out.println("Child: " + value); // Own value
        System.out.println("Parent.value: " + super.value); // Parent's value
    }
}
```

`super` is like `this` but refers to the parent class.

Slide 8: Superclass Constructor Call

Chaining constructors with `super()`:

```
class Vehicle {
    String brand;

    Vehicle(String brand) {
        this.brand = brand;
        System.out.println("Vehicle constructor");
    }
}

class Car extends Vehicle {
    String model;

    Car(String brand, String model) {
        super(brand);    // Must be FIRST statement
        this.model = model;
        System.out.println("Car constructor");
    }
}

// new Car("Toyota", "Camry")
// Output:
// Vehicle constructor
// Car constructor
```

Slide 9: Constructor Call Order

When a derived object is created:

```
class A {
    A() { System.out.println("A"); }
}
```

```
class B extends A {
    B() { super(); System.out.println("B"); }    // super() is implicit
}

class C extends B {
    C() { System.out.println("C"); }    // super() call to B() is implicit
}

new C();
// Output:
// A
// B
// C
```

Order: Parent → Child (top to bottom)

- Java always calls the parent constructor before the child's body
- If `super()` is not written explicitly, the compiler inserts `super()` (no-arg)

Slide 10: Implicit super() Call

Compiler inserts super() automatically:

```
class Parent {
    Parent() {
        System.out.println("Parent constructor");
    }
}

class Child extends Parent {
    Child() {
        // super();          ← Compiler inserts this automatically
        System.out.println("Child constructor");
    }
}
```

Warning: If the parent has NO no-arg constructor, you MUST call `super(args)` explicitly:

```
class Parent {
    Parent(int x) { }    // Only parameterized constructor
}

class Child extends Parent {
    Child() {
        super(10);        // Must call explicitly!
    }
}
```

```
}  
}
```

Slide 11: Constructor Chain in Deep Hierarchy

```
class Grandparent {  
    Grandparent() {  
        System.out.println("1. Grandparent constructor");  
    }  
}  
  
class Parent extends Grandparent {  
    Parent() {  
        System.out.println("2. Parent constructor");  
    }  
}  
  
class Child extends Parent {  
    Child() {  
        System.out.println("3. Child constructor");  
    }  
}  
  
new Child();  
// Output:  
// 1. Grandparent constructor  
// 2. Parent constructor  
// 3. Child constructor
```

Each constructor calls `super()` first, creating a chain from **Object** → **top-level parent** → **bottom child**.

Slide 12: Method Overriding

Redefining a parent method in the child:

```
class Animal {  
    void speak() {  
        System.out.println("Animal speaks");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void speak() { // Override parent's speak()  
        System.out.println("Woof!");  
    }  
}
```

```
class Cat extends Animal {
    @Override
    void speak() {           // Override parent's speak()
        System.out.println("Meow!");
    }
}
```

Same method signature, different implementation in each subclass.

Slide 13: The `@Override` Annotation

Why use it?

```
class Parent {
    void display() { }
}

class Child extends Parent {
    @Override
    void display() { }        // Compiler checks this actually overrides

    @Override
    void show() { }          // COMPILE ERROR: no such method in Parent
}
```

- Not required, but **strongly recommended**
 - Compiler warns if the method doesn't actually override anything
 - Catches typos: `disply()` instead of `display()`
-

Slide 14: Overriding Rules

Method must have:

- Same **name**
- Same **parameter list**
- Same **return type** (or covariant return type)
- Same or **more accessible** access modifier
- Cannot be `private`, `static`, or `final`

```
class Parent {
    protected Number getValue() { return 10; }
}

class Child extends Parent {
    @Override
```

```
public Integer getValue() { return 20; } // OK: wider access, covariant
return
}
```

- `protected` → `public`: ✓ (wider access allowed)
- `Number` → `Integer`: ✓ (covariant return — subclass of `Number`)

Slide 15: Covariant Return Type

Return type can be a subclass of the original:

```
class Shape {
    Shape clone() { return new Shape(); }
}

class Circle extends Shape {
    @Override
    Circle clone() { return new Circle(); } // Return Circle, not Shape
}

// Usage:
Shape s = new Circle();
Circle c = ((Circle) s).clone(); // No cast needed in Java 5+
```

- Introduced in Java 5
- Eliminates unnecessary downcasting

Slide 16: Calling the Overridden Method

Using `super` to call parent version:

```
class Employee {
    double calculatePay() {
        return 0;
    }

    String getInfo() {
        return "Employee";
    }
}

class Manager extends Employee {
    private int teamSize;

    @Override
    double calculatePay() {
        return super.calculatePay() + teamSize * 100; // Extend parent
    }
}
```



```
    }

    @Override
    String getInfo() {
        return super.getInfo() + " (Manager)";    // Extend parent
    }
}
```

- `super.method()` calls the parent version
- Used for **extending** (adding to parent behavior)
- Without `super`, the method is completely **replaced**

Slide 17: Forbidden Override — final Methods

Preventing overriding with `final`:

```
class Parent {
    final void cannotOverride() {
        System.out.println("This method is final");
    }

    void canOverride() {
        System.out.println("This can be overridden");
    }
}

class Child extends Parent {
    // void cannotOverride() { }    // COMPILE ERROR: cannot override final

    @Override
    void canOverride() {            // This is fine
        System.out.println("Overridden!");
    }
}
```

- `final` methods cannot be overridden
- Used for critical implementation details (security, framework hooks)

Slide 18: Forbidden Override — static Methods

Static methods belong to the **CLASS**, not the object:

```
class Parent {
    static void whoAmI() {
        System.out.println("Parent");
    }
}
```

```
class Child extends Parent {
    // HIDES Parent.whoAmI(), does NOT override it
    static void whoAmI() {
        System.out.println("Child");
    }
}

Parent p = new Child();
p.whoAmI();           // "Parent" – static methods are NOT polymorphic
```

Static methods can be **hidden** but not overridden. Method resolution is at compile-time based on reference type. Attempting to place `@Override` on a static method forces a compile-time error — this is the standard way to detect and prevent accidental hiding.

Slide 19: Abstract Classes

Defining a template for subclasses:

```
abstract class Shape {
    protected String color;

    Shape(String color) {
        this.color = color;
    }

    abstract double area();           // No body – must override
    abstract double perimeter();      // No body – must override

    String getColor() {               // Concrete method – already implemented
        return color;
    }
}

class Circle extends Shape {
    private double radius;

    Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    double area() { return Math.PI * radius * radius; }

    @Override
    double perimeter() { return 2 * Math.PI * radius; }
}
```

```
// Shape s = new Shape("red");    // COMPILE ERROR: abstract class
Circle c = new Circle("blue", 5); // OK – all abstract methods implemented
```

Slide 20: Abstract vs Concrete Methods

Feature	Abstract method	Concrete method
Has body	No	Yes
Must override?	Yes (in first concrete subclass)	No (optional)
Keyword	<code>abstract</code>	(none)
In abstract class	✓	✓
In concrete class	✗	✓

```
abstract class Database {
    abstract void connect();    // Subclass must implement

    void disconnect() {        // Optional override
        System.out.println("Disconnected");
    }
}
```

Slide 21: Interface Inheritance

Pure abstraction via interfaces:

```
interface Flyable {
    void fly();                // public abstract by default
}

interface Swimmable {
    void swim();
}

// A class can implement MULTIPLE interfaces
class Duck implements Flyable, Swimmable {
    @Override
    public void fly() {
        System.out.println("Duck flying");
    }

    @Override
    public void swim() {
        System.out.println("Duck swimming");
    }
}
```

```

    }
}

```

- Interfaces are **fully abstract** (Java 7 and earlier)
- Java 8+ allows **default** and **static** methods in interfaces
- A class can **extends** ONE class but **implements** MANY interfaces
- Interfaces can also extend multiple interfaces: **interface C extends A, B { }**

Slide 22: Default Methods in Interfaces (Java 8+)

```

interface Vehicle {
    void start();                // Abstract

    default void honk() {        // Default implementation
        System.out.println("Beep!");
    }

    static boolean isValid(Vehicle v) { // Static utility
        return v != null;
    }
}

class Car implements Vehicle {
    @Override
    public void start() {
        System.out.println("Car starting");
    }
    // honk() inherited with default implementation
}

Car c = new Car();
c.start();    // Car starting
c.honk();     // Beep!
Vehicle.isValid(c); // true

```

Slide 23: The Diamond Problem (Interfaces)

Java solves it with explicit override:

```

interface A {
    default void show() { System.out.println("A"); }
}

interface B {
    default void show() { System.out.println("B"); }
}

```

```
class Child implements A, B {
    // Must override – otherwise compile error!
    @Override
    public void show() {
        A.super.show();    // Explicitly choose A's version
        B.super.show();    // Or call B's version
        System.out.println("Child");
    }
}
```

No ambiguity in Java — the compiler forces you to resolve the conflict.

Slide 24: Polymorphism with Inheritance

Treating objects by their parent type:

```
class Animal {
    void speak() { System.out.println("Some sound"); }
}

class Dog extends Animal {
    @Override
    void speak() { System.out.println("Woof!"); }
    void bark() { System.out.println("Barking..."); }
}

class Cat extends Animal {
    @Override
    void speak() { System.out.println("Meow!"); }
}

// Polymorphic array
Animal[] animals = {new Dog(), new Cat(), new Dog()};
for (Animal a : animals) {
    a.speak();           // Correct method called for each type
}

// Output:
// Woof!
// Meow!
// Woof!
```

Slide 25: Polymorphism (continued)

Compile-time vs Runtime type:

```
Animal a = new Dog();    // Reference: Animal, Object: Dog
a.speak();               // "Woof!" – runtime method lookup
```

```
// a.bark();           // COMPILE ERROR: Animal has no bark()

if (a instanceof Dog) {
    Dog d = (Dog) a;    // Downcast to access Dog-specific methods
    d.bark();
}
```

Concept	Behavior
Reference type	Determines which members are accessible
Object type	Determines which method body runs
instanceof	Checks runtime type before casting

Slide 26: The final Keyword in Inheritance

Three uses of final:

```
final class Constants {           // Cannot be extended
    static final double PI = 3.14159; // Cannot be reassigned
}

class Parent {
    final void secureMethod() { } // Cannot be overridden
}

// class Extended extends Constants { } // COMPILE ERROR: final class
```

Context	Effect
final class	Cannot be subclassed
final method	Cannot be overridden
final variable	Cannot be reassigned (constant)

Note: Java 15+ introduces **sealed classes** as a more granular alternative to final. A sealed class explicitly permits specific subclasses while restricting all others: sealed class Shape permits Circle, Rectangle { }

Slide 27: Complete Example — Employee Hierarchy

```
abstract class Employee {
    protected String empId;
    protected String name;

    Employee(String empId, String name) {
```

```
        this.empId = empId;
        this.name = name;
    }

    abstract double calculatePay();

    String getInfo() {
        return empId + ": " + name;
    }
}

class HourlyEmployee extends Employee {
    private double hourlyRate;
    private int hoursWorked;

    HourlyEmployee(String id, String name, double rate, int hours) {
        super(id, name);
        this.hourlyRate = rate;
        this.hoursWorked = hours;
    }

    @Override
    double calculatePay() {
        return hourlyRate * hoursWorked;
    }
}
```

Slide 28: Complete Example (continued)

```
class SalariedEmployee extends Employee {
    private double annualSalary;

    SalariedEmployee(String id, String name, double salary) {
        super(id, name);
        this.annualSalary = salary;
    }

    @Override
    double calculatePay() {
        return annualSalary / 12;    // Monthly pay
    }
}

// Usage
Employee[] workers = {
    new HourlyEmployee("E001", "Alice", 20, 160),
    new SalariedEmployee("E002", "Bob", 60000)
};

for (Employee emp : workers) {
```

```
        System.out.println(emp.getInfo() +
                           ": $" + emp.calculatePay());
    }
    // E001: Alice: $3200.0
    // E002: Bob: $5000.0
```

Slide 29: Common Mistakes

1. Forgetting `super()` in constructor:

```
class Parent {
    Parent(String name) { }
}

class Child extends Parent {
    Child() {
        // super("name");    // Missing!
    } // COMPILE ERROR: no default constructor in Parent
}
```

2. Weaker access in override:

```
class Parent {
    protected void method() { }
}

class Child extends Parent {
    @Override
    void method() { } // COMPILE ERROR: weaker access (protected → default)
}
```

Slide 30: Common Mistakes (continued)

3. Confusing hide and override:

```
class Parent {
    static void staticMethod() { System.out.println("Parent"); }
}

class Child extends Parent {
    static void staticMethod() { System.out.println("Child"); } // Hides, not
    overrides
}
```



```
Parent p = new Child();
p.staticMethod();    // "Parent" – static, NOT polymorphic!
```

4. Calling overridable method from constructor:

```
class Parent {
    Parent() { init(); }           // Calls Child's init() before Child's
    constructor!
    void init() { System.out.println("Parent init"); }
}

class Child extends Parent {
    private int value = 42;
    @Override
    void init() { System.out.println("Child init: " + value); } // value still 0!
}

new Child(); // "Child init: 0" – field not yet initialized!
```

Slide 31: Best Practices

Do's:

- ✓ Favor composition over inheritance ("has-a" vs "is-a")
- ✓ Declare methods `final` if not designed for override
- ✓ Use `@Override` annotation always
- ✓ Keep hierarchy shallow (≤ 3 levels)
- ✓ Use abstract classes for shared state, interfaces for behavior
- ✓ Make constructors `protected` or package-private for abstract classes

Don'ts:

- ✗ Call overridable methods from constructors
- ✗ Use deep inheritance chains
- ✗ Override just to throw an exception (breaks LSP)
- ✗ Make fields `public` in base classes
- ✗ Forget `super()` in constructor chain

Slide 32: Summary

Inheritance:

- `class Child extends Parent` — single inheritance only
- All classes inherit from `Object`
- Fields are inherited; methods can be overridden

Overriding:

- Same signature, `@Override` annotation
- Can call parent version with `super.method()`
- Abstract methods MUST be overridden
- `final` methods cannot be overridden

Constructor Call Order:

- Parent constructor runs BEFORE child's body
- Chain: Object → top parent → ... → Child
- `super(args)` must be first statement in constructor
- If no explicit `super()`, compiler inserts `super()` (no-arg)

Key Takeaway: Inheritance is essential for code reuse and polymorphism in Java, but use it judiciously. Prefer interfaces for behavioral contracts and abstract classes for partial implementations with shared state.

Slide 33: Resources & Further Reading

Official Documentation:

- Java Inheritance: docs.oracle.com/javase/tutorial/java/landl/
- Overriding: docs.oracle.com/javase/tutorial/java/landl/override.html
- Polymorphism: docs.oracle.com/javase/tutorial/java/landl/polymorphism.html
- Abstract Methods: docs.oracle.com/javase/tutorial/java/landl/abstract.html

Topics for Further Study:

- Generics and inheritance (`? extends T`, `? super T`)
- Sealed classes (Java 17+)
- Record classes and inheritance
- Composition vs Inheritance deep dive
- Design patterns (Template Method, Strategy, Factory)

Thank You!

Questions?